

Universidad Nacional de Lomas de Zamora

Facultad de Ingeniería

Proyecto FINAL - Estacion de calidad por Perfilometria

Alumno: QUINTANA, Fernando Miguel

In []:

```

1  import numpy as np
2
3  # -----
4  # GeometrySpace
5  # -----
6
7  # Planos y rectas en el espacio euclídeo 3D
8  # https://www2.eii.uva.es/fund_inf/python/notebooks/Bibliotecas/03_Numpy/Numpy.html
9
10 def normaliza(v, tol=1e-6):
11     """
12     Devuelve el vector normalizado de v.
13
14     Parameters
15     -----
16     v : list, tuple, numpy.ndarray: numérico
17         Vector unidimensional
18     tol : float
19         Tolerancia que debe cumplir la norma del vector v
20     Returns
21     -----
22     v_norm : numpy.ndarray of floats
23         Vector normalizado unitario
24     Raises
25     -----
26     ValueError
27         Si la norma del vector inicial es inferior a tol
28     Example
29     -----
30         >>> v = (1, 1, 1)
31         >>> normaliza(v)
32     """
33
34     norma = np.linalg.norm(v)
35     if norma < tol:
36         raise ValueError("La norma del vector es inferior a la tolerancia {}".format(tol))
37     v_norm = v / norma # Al usar np.linalg.norm() v_norm es un numpy.ndarray
38     return v_norm
39
40
41 def distancia_entre_puntos(p1, p2):

```

```

42     """
43     Calcula la distancia entre dos puntos en el espacio.
44
45     Parameters
46     -----
47     p1, p2 : numpy.ndarray
48         Vectores unidimensionales
49     Returns
50     -----
51     dist : numpy.float
52         Distancia entre los puntos p1 y p2
53     Example:
54     -----
55     >>> p1 = np.array((1, 1, 1), dtype=float)
56     >>> p2 = np.array((2, 3, 4), dtype=float)
57     >>> distancia_entre_puntos(p1, p2)
58     """
59     dist = np.linalg.norm(p1 - p2) # p1-p2 no está definido para tuplas o listas
60     return dist
61
62
63 def angulo(v1, v2):
64     """
65     Calcula el ángulo en radianes entre dos vectores.
66
67     Parameters
68     -----
69     v1, v2 : tuple, list, numpy.ndarray: numérico
70         Vectores unidimensionales
71     Returns
72     -----
73     angulo : float
74         Ángulo en radianes que forman los vectores
75     Example
76     -----
77     >>> u_x = [1, 0, 0] # Vector unitario eje X
78     >>> u_y = [0, 1, 0] # Vector unitario eje Y
79     >>> angulo(u_x, u_y)*180/np.pi # Ángulo en grados
80     """
81
82     prod_esc = np.dot(v1, v2)
83     norma_v1 = np.linalg.norm(v1)

```

```

84     norma_v2 = np.linalg.norm(v2)
85
86     angulo = np.arccos(np.clip(prod_esc / (norma_v1 * norma_v2), -1, 1))
87     return angulo
88
89
90 def plano_tres_puntos(p1, p2, p3, tol=1e-3):
91     """
92     Calcula el plano definido por tres puntos en el espacio.
93
94     Parameters
95     -----
96     p1, p2, p3 : numpy.ndarray of floats
97         Vectores unidimensionales representando 3 puntos 3D
98     tol : float
99         Tolerancia que deben cumplir las distancias entre los puntos
100     Returns
101     -----
102     n, D : tuple : (numpy.ndarray of float, numpy float)
103         n es el vector unitario perpendicular al plano
104         D es el término independiente de la ecuación algebraica Ax+By+Cz+D=0
105     Raises
106     -----
107     ValueError
108         Si la distancia entre alguno de los 3 puntos es inferior a tol
109     Example
110     -----
111     >>> p1 = np.array((1, 0, 0))
112     >>> p2 = np.array((0, 1, 0))
113     >>> p3 = np.array((0, 0, 1))
114     >>> n, D = plano_tres_puntos(p1, p2, p3)
115     """
116     dist12 = distancia_entre_puntos(p1, p2)
117     dist13 = distancia_entre_puntos(p1, p3)
118     dist23 = distancia_entre_puntos(p2, p3)
119
120     if dist12 < tol or dist13 < tol or dist23 < tol:
121         raise ValueError("Distancia entre puntos menor que la tolerancia {}".format(tol))
122
123     # Producto vectorial para A B C
124     # | i      j      k      |
125     # |v_12[0] v_12[1] v_12[2]|

```

```

126     # |v_13[0] v_13[1] v_13[2]|
127
128     v12 = p1 - p2
129     v13 = p1 - p3
130
131     n = normaliza(np.cross(v12, v13))
132
133     D = -np.dot(n, p1)
134
135     return n, D
136
137
138 def recta_dos_puntos(p1, p2, tol=1e-3):
139     """
140     Parameters
141     -----
142     p1, p2 : numpy.ndarray of floats
143         Vectores unidimensionales representando 2 puntos 3D
144     tol : float
145         Tolerancia que debe cumplir la distancia entre los puntos
146     Returns
147     -----
148     p, v : tuple: (numpy.ndarray of float, numpy.ndarray of float)
149         p es una copia de p1, punto pasado como primer parámetro
150
151         v es el vector director unitario de la recta
152     Raises
153     -----
154     ValueError
155         Si la distancia entre los 2 puntos es inferior a tol
156     Example
157     -----
158     >>> p1 = np.array((2, 2, 2))
159     >>> p2 = np.array((3, 3, 3))
160     >>> p, v = recta_dos_puntos(p1, p2)
161     """
162     if distancia_entre_puntos(p1, p2) < tol:
163         raise ValueError("Distancia entre puntos menor que la tolerancia {}".format(tol))
164     v = normaliza(p1 - p2)
165     return np.copy(p1), v
166
167

```

```

168 def interseccion_recta_plano(recta, plano, tol=0.0005):
169     """
170     Parameters
171     -----
172     recta : tuple : (numpy.ndarray of floats, numpy.ndarray of floats)
173         Recta definida por la tupla punto inicial y vector director
174         p=p_0+l*v
175     plano : tuple : (numpy.ndarray of floats, float)
176         Plano definido por la tupla vector normal (A, B, C) y término independiente D
177         Ax+By+Cz+D=0
178     tol : float
179         Tolerancia que deben cumplir el ángulo entre recta y plano
180     Returns
181     -----
182     p : numpy.ndarray of floats
183         El punto de intersección
184     Raises
185     -----
186     ValueError
187         Si el ángulo entre ambas figuras es inferior a tol en radianes
188     Example
189     -----
190     >>> p1 = np.array((1, 0, 0))
191     >>> p2 = np.array((0, 1, 0))
192     >>> p3 = np.array((0, 0, 1))
193     >>> p4 = np.array((2, 2, 2))
194     >>> p5 = np.array((3, 3, 3))
195     >>> plano = plano_tres_puntos(p1, p2, p3)
196     >>> recta = recta_dos_puntos(p4, p5)
197     >>> p = interseccion_recta_plano(recta, plano)
198     """
199     ang = angulo(recta[1], plano[0])
200
201     if np.abs(ang - np.pi / 2) < tol:
202         print(np.abs(ang - np.pi / 2))
203         raise ValueError("La recta y el plano son casi paralelos según tolerancia {}".format(tol))
204
205     lamb = -(np.dot(recta[0], plano[0]) + plano[1]) / np.dot(recta[1], plano[0])
206
207     return recta[0] + lamb * recta[1]
208
209

```

```

210 # -----
211 # GeneradorMatriz
212 # -----
213
214 # punto focal
215 p_focal = np.array((-96.10, 166.45, 0))
216
217 # punto plano del Laser
218 p_laser_1 = np.array((0, 0, 0))
219 p_laser_2 = np.array((0, 1, 0))
220 p_laser_3 = np.array((0, 0, 1))
221
222 # Datos cámara
223 px_wh = 0.003      # mm tamaño de alto - ancho de cada pixel
224 focal_dist = 7.8    # mm distancia focal de cámara
225 W = 732            # Referencia pixeles verticales -> range(0, 733):
226 H = 1296           # Referencia pixeles horizontales -> range(-648, 649):
227
228 # Creamos el plano del Láser
229 plano_laser = plano_tres_puntos(p1=p_laser_1, p2=p_laser_2, p3=p_laser_3)
230
231 # Recorremos cada pixel del sensor de la cámara
232 mtx_ppx = []
233 mtx_ppy = []
234 mtx_ppz = []
235
236 for fpx in range(0, 733):
237     aux_x = []
238     aux_y = []
239     aux_z = []
240
241     # Posición de cada pixel en X e Y (varía con la fila)
242     pos_x = fpx * px_wh * np.cos(30 * np.pi / 180) - (100 + 360 * px_wh * np.cos(30 * np.pi / 180))
243     pos_y = fpx * px_wh * np.sin(30 * np.pi / 180) + (173.21 - 360 * px_wh * np.sin(30 * np.pi / 180))
244
245     # Recorremos por columna del sensor
246     for cpx in range(-648, 649):
247         if cpx == 0:
248             pass
249         else:
250             pos_z = cpx * px_wh
251

```



```
252     # Creamos la recta del Laser
253     px_pos = np.array((pos_x, pos_y, pos_z))
254     recta_haz = recta_dos_puntos(px_pos, p_focal)
255
256     # Obtenemos intersección recta-plano
257     intersection = interseccion_recta_plano(recta=recta_haz, plano=plano_laser)
258
259     # Guardamos la posición X,Y,Z en el espacio para ese pixel
260     XX = np.around(intersection, 3).tolist()[0]
261     YY = np.around(intersection, 3).tolist()[1]
262     ZZ = np.around(intersection, 3).tolist()[2]
263
264     #print(px_pos)
265     print("Para el pixel en el punto " + str(px_pos) + " --- > " + str(XX) + ";" + str(YY) + ";" + str(ZZ))
266     aux_x.append(XX)
267     aux_y.append(YY)
268     aux_z.append(ZZ)
269
270     mtx_ppx.append(aux_x)
271     mtx_ppy.append(aux_y)
272     mtx_ppz.append(aux_z)
273
274     np.savetxt("mtx_x_izq.txt", mtx_ppx)
275     np.savetxt("mtx_y_izq.txt", mtx_ppy)
276     np.savetxt("mtx_z_izq.txt", mtx_ppz)
277
```

In []:

1